F1TENTH Autonomous Racing — ROS 2 Jazzy

A complete autonomous racing stack for the F1TENTH simulator, running natively on Ubuntu 24.04 + ROS 2 Jazzy. Includes a high-performance Stanley controller, SLAM mapping, automated raceline generation, and an experimental head-to-head overtaking pipeline.

Table of Contents

1. Initial Setup & Dependencies
2. Quick Start — Launch & Race
3. Racing on a New Map
4. Raceline Generation Pipeline — What Each Step Does
5. Automated Parameter Tuning (Optuna)
6. Controller Details
7. Head-to-Head Overtaking (⚠️ Work In Progress)
8. Configuration Reference

---

1. Initial Setup & Dependencies

System Requirements

- **OS**: Ubuntu 24.04
- **ROS 2**: Jazzy Jalisco (installed at `/opt/ros/jazzy/`)
- **Python**: 3.12+

ROS 2 Packages (install via apt)

```
sudo apt install \
  ros-jazzy-nav2-map-server \
  ros-jazzy-nav2-lifecycle-manager \
  ros-jazzy-slam-toolbox \
  ros-jazzy-robot-state-publisher \
  ros-jazzy-xacro \
  ros-jazzy-rviz2 \
  ros-jazzy-foxglove-bridge
```

Python Dependencies

```
pip3 install numpy scipy matplotlib Pillow optuna --break-system-packages
```

F1TENTH Gym (simulator core)

The simulator physics engine is installed as an editable Python package:

```
cd ~/f1tenth_ws
pip3 install -e ./src/f1tenth_gym_pkg --break-system-packages
```

> ***Note:*** *This package includes a compatibility guard for the `coverage` / `numba` combination on Ubuntu 24.04.*

Build the Workspace

```
cd ~/f1tenth_ws
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
```

You need to rebuild after editing any Python node source files. Config YAML changes do **not** require a rebuild (just restart the node).

---

2. Quick Start — Launch & Race

This gets the car racing on the default **Spielberg** track in under 30 seconds.

Terminal 1 — Launch the Simulator

```
cd ~/f1tenth_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch f1tenth_gym_ros gym_bridge_launch.py launch_rviz:=true
```

You should see the track load in RViz with the car at the origin.

Terminal 2 — Launch the Stanley Controller

```
cd ~/f1tenth_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run gap_follow stanley_node
```

The car will immediately start racing the optimal line. The Stanley controller is the recommended controller — it directly minimizes cross-track error and uses feedforward steering for precise cornering.

Terminal 3 — Lap Timer (Optional)

```
cd ~/f1tenth_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 run gap_follow lap_timer_node
```

Prints lap times, best time, and average speed to the console. Press `Ctrl+C` for a session summary.

Verify It's Working

```
ros2 topic list --no-daemon
```

You should see `/drive`, `/ego_racecar/odom`, `/ego_racecar/scan`, `/map`, and `/tf` among others.

---

3. Racing on a New Map

This section covers the complete workflow for taking a new track map and getting the car to race on it at full speed.

3.1 — Loading a Downloaded Map

If you already have map files (`.png` + `.yaml`) from the internet:

1. Copy them into the maps directory:

```
cp my_track.png ~/f1tenth_ws/src/f1tenth_gym_ros/maps/
cp my_track.yaml ~/f1tenth_ws/src/f1tenth_gym_ros/maps/
```

2. Make sure the `.yaml` file's `image:` field points to the correct `.png` filename.

3. Tell the simulator to use the new map — edit `src/f1tenth_gym_ros/config/sim.yaml` :

```
map_path: 'maps/my_track'
```

4. Set the ego starting pose in `sim.yaml` to a valid position on the new track:

```
sx: 0.0
sy: 0.0
stheta: 0.0
```

5. Rebuild and relaunch:

```
colcon build --symlink-install
ros2 launch f1tenth_gym_ros gym_bridge_launch.py launch_rviz:=true
```

3.2 — Creating a Map with SLAM

If you need to map an unknown track using the car's LiDAR:

Terminal 1 — Launch SLAM mapping stack:

```
cd ~/f1tenth_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
ros2 launch f1tenth_gym_ros slam_mapping_launch.py
```

This launches the simulator (single agent, teleop enabled), SLAM Toolbox in async mapping mode, and RViz. The LiDAR scan is automatically remapped from `/ego_racecar/scan` to the SLAM node.

Terminal 2 — Drive the car manually:

```
cd ~/f1tenth_ws
source /opt/ros/jazzy/setup.bash
```

```
source install/setup.bash
ros2 run f1tenth_gym_ros teleop_ackermann_node --ros-args -p max_forward_speed:=2.0
```

Teleop Controls:

Key	Action
`W` / `↑`	Accelerate forward
`S` / `↓`	Brake / reverse
`A` / `←`	Steer left
`D` / `→`	Steer right
`C`	Center steering
`Space`	Full stop
`Q`	Quit

Drive slowly around the entire track. Watch the map build in RViz.

Terminal 3 — Save the map (once the map looks complete in RViz):

```
cd ~/f1tenth_ws/src/f1tenth_gym_ros/maps
ros2 run nav2_map_server map_saver_cli -f my_track
```

This creates `my_track.png` and `my_track.yaml` in the maps directory. You can now close the SLAM and teleop terminals.

3.3 — Generate the Optimal Raceline

Before the car can race, it needs to know the fastest geometric path through the track and the maximum safe speed at every point. Run:

```
cd ~/f1tenth_ws
./generate_raceline.sh my_track
```

(If not executable: `chmod +x generate_raceline.sh`)

This runs a 4-step pipeline explained in detail in Section 4. When it finishes, check `raceline_visualization.png` — you should see a color-coded racing line overlaid on your track (green = fast, red = slow).

Output files:

File	Location	Purpose
`my_track_centerline.csv`	`maps/`	Extracted track centerline
`my_track_raceline.csv`	`maps/`	Optimized raceline with speed profile
`raceline.csv`	`maps/`	Copy of the above (used by controllers)

`raceline_visualization.png`	workspace root	Visual preview

3.4 — Tune Controller Parameters (Optional but Recommended)

Every track has different characteristics. The Optuna tuner runs hundreds of headless simulated laps to find optimal Stanley parameters for your specific track. See Section 5.

3.5 — Code Changes Needed

After generating the raceline, update these files:

1. **`src/f1tenth_gym_ros/config/sim.yaml`** — Change the map:

   ```
   map_path: 'maps/my_track'
   ```

   Also update `sx`, `sy`, `stheta` to valid starting coordinates on the new track.

2. **Rebuild the workspace:**

   ```
   cd ~/f1tenth_ws
   colcon build --symlink-install
   ```

3. **`raceline.csv` is auto-updated** — The `generate_raceline.sh` script automatically copies the new raceline to `maps/raceline.csv`, which is the default path all controller nodes read. No code changes needed in the nodes themselves.

3.6 — Race on the New Map

```
# Terminal 1 — Simulator
ros2 launch f1tenth_gym_ros gym_bridge_launch.py launch_rviz:=true

# Terminal 2 — Stanley Controller
ros2 run gap_follow stanley_node

# Terminal 3 — Lap Timer (optional)
ros2 run gap_follow lap_timer_node
```

---

4. Raceline Generation Pipeline — What Each Step Does

The `generate_raceline.sh` script automates 4 steps. Here's what happens under the hood:

Step 1: Centerline Extraction (`extract_centerline.py`)

What it does: Takes the map image (`.png`) and extracts the drivable track centerline using computer vision.

How it works:

- Reads the map image and its YAML metadata (resolution, origin)
- Flood-fills from the world origin `(0, 0)` to isolate only the inner track area (ignoring outside boundaries)

- Skeletonizes the drivable area down to a 1-pixel-wide spine
- Orders the skeleton points into a continuous loop via nearest-neighbor traversal
- Fits a periodic B-spline (smoothing factor 2.0) and resamples at 0.4m intervals
- Computes track widths at each point using the distance transform, smoothed and clamped to 1.0–1.2m
- Saves a preview image to `centerline_preview.png`

Output: `maps/<map_name>_centerline.csv` — columns: `x_m, y_m, w_tr_right_m, w_tr_left_m`

Step 2: Global Trajectory Optimization (`main_globaltraj.py`)

What it does: Computes the geometrically optimal racing line — the shortest/fastest path that stays within the track boundaries.

How it works:

- Uses the TUM Global Racetrajectory Optimization tool (the `global_racetrajectory_optimization/` directory)
- Takes the centerline + track widths as input
- Solves a minimum-curvature optimization problem: finds the path that minimizes total curvature (and therefore maximizes possible speed through corners)
- The optimizer uses quadratic programming to shift the raceline left/right within the track boundaries

Output: `global_racetrajectory_optimization/outputs/traj_race_cl.csv` — the optimized geometric path with curvature at each point.

Step 3: Speed Profile Generation (`speed_profile.py`)

What it does: Calculates the maximum safe velocity at every point on the raceline based on physics.

How it works — three passes:

1. **Curvature limit:** `v = sqrt(A_LAT_MAX / kappa)` — tighter corners → slower speed
2. **Forward acceleration limit:** Starting from each point, limits how fast the car can accelerate given `A_LON_MAX`
3. **Backward braking limit:** Working backwards from each point, limits speed based on braking capability `A_LON_BRAKE`

Vehicle parameters (in `speed_profile.py`):

Parameter	Value	Meaning
`V_MAX`	8.0 m/s	Absolute top speed
`A_LAT_MAX`	7.0 m/s²	Max lateral acceleration (tire grip)
`A_LON_MAX`	5.0 m/s²	Max acceleration (VESC motor limit)
`A_LON_BRAKE`	9.0 m/s²	Max braking deceleration

Output: `maps/<map_name>_raceline.csv` — the final raceline with columns: `s_m; x_m; y_m; psi_rad; kappa_radpm; vx_mps; ax_mps2`

Also automatically copies to `maps/raceline.csv` for backward compatibility with all controller nodes.

Step 4: Visualization (`visualize_raceline.py`)

What it does: Generates `raceline_visualization.png` showing the raceline overlaid on the map, color-coded by speed:

- 🔴 **Red** = slow corners
- 🟢 **Green** = full speed straights
- ⚪ **Dashed white** = centerline (for comparison)

You can re-run visualization standalone at any time:

```
python3 visualize_raceline.py --map_name my_track
```

---

5. Automated Parameter Tuning (Optuna)

The `stanley_tuner.py` script uses Bayesian optimization (Optuna) to find the best `k_gain` , `k_soft` , and `speed_scale` for any track.

Setup for a New Track

Edit `stanley_tuner.py` and update:

```
config = EnvConfig(map_name='my_track', num_agents=1, render_enabled=False)
script_dir = os.path.dirname(os.path.abspath(__file__))
raceline_path = os.path.join(script_dir, 'src', 'f1tenth_gym_ros', 'maps',
'my_track_raceline.csv')
```

Run

```
cd ~/f1tenth_ws
python3 stanley_tuner.py
```

Takes ~2 minutes. It runs 100 headless trials across multiple CPU cores and prints the best parameters:

```
Best parameters:
  k_gain: 2.15
  k_soft: 0.40
  speed_scale: 1.08
Best lap time: 40.14 seconds
```

Apply Results

Update the defaults in `src/gap_follow/gap_follow/stanley_node.py` :

```
self.declare_parameter('k_gain',      2.15)   # from Optuna
self.declare_parameter('k_soft',      0.40)   # from Optuna
self.declare_parameter('speed_scale', 1.08)   # from Optuna
```

Then rebuild: `colcon build --symlink-install`

---

6. Controller Details

Stanley Controller (`stanley_node`) — Recommended

The Stanley controller directly minimizes cross-track error for precise raceline tracking. It outperforms Pure Pursuit on tight corners because it doesn't cut them.

How it works:

- **Cross-track error:** `e_fa = sin(ψ) · dx - cos(ψ) · dy` — exact lateral deviation from the path at the front axle
- **Feedforward steering:** `arctan(wheelbase × κ)` — uses precomputed track curvature to anticipate corners before entering them
- **Speed-softening:** `arctan2(k · e_fa, speed + k_soft)` — prevents oscillation at low speeds
- **Latency compensation:** Reads target speed from 3 waypoints ahead to account for actuator delay

Parameters:

Parameter	Default	Description
`k_gain`	`2.0`	Cross-track correction strength
`k_soft`	`0.37`	Low-speed softening
`speed_scale`	`1.12`	Multiplier on raceline speed
`wheelbase`	`0.33`	Car wheelbase (m)
`steering_max`	`0.41`	Max steering angle (rad)

Publishes: `/drive` , `/stanley/markers` **Subscribes:** `/ego_racecar/odom`

(Default parameters are optimized for Spielberg, achieving ~40.14s laps.)

Pure Pursuit Controller (`pure_pursuit_node`) — Legacy

A lookahead-based controller. Simpler but cuts corners at high speed.

- **Adaptive lookahead:** Scales from 0.7m to 1.8m with speed
- **Circle-line interpolation:** Exact intersect on waypoint segments for smoother tracking
- **Latency compensation:** Reads speed from 3 waypoints ahead
- **Safety limiter:** Half speed when heading error exceeds 45°
- **RViz markers:** Publishes lookahead circle + target on `/pure_pursuit/markers`

```
ros2 run gap_follow pure_pursuit_node --ros-args -p speed_scale:=0.3
```

Lap Timer (`lap_timer_node`)

- Tracks nearest waypoint index along the raceline
- Detects lap completion by waypoint index wrap-around (>80% → <20%)
- Ignores out-lap until first start/finish crossing
- 10-second debounce prevents double-triggering
- Reports: lap time, best time, average time, average speed

- Session summary on `Ctrl+C`

Publishes: `/lap_timer/info` (String)

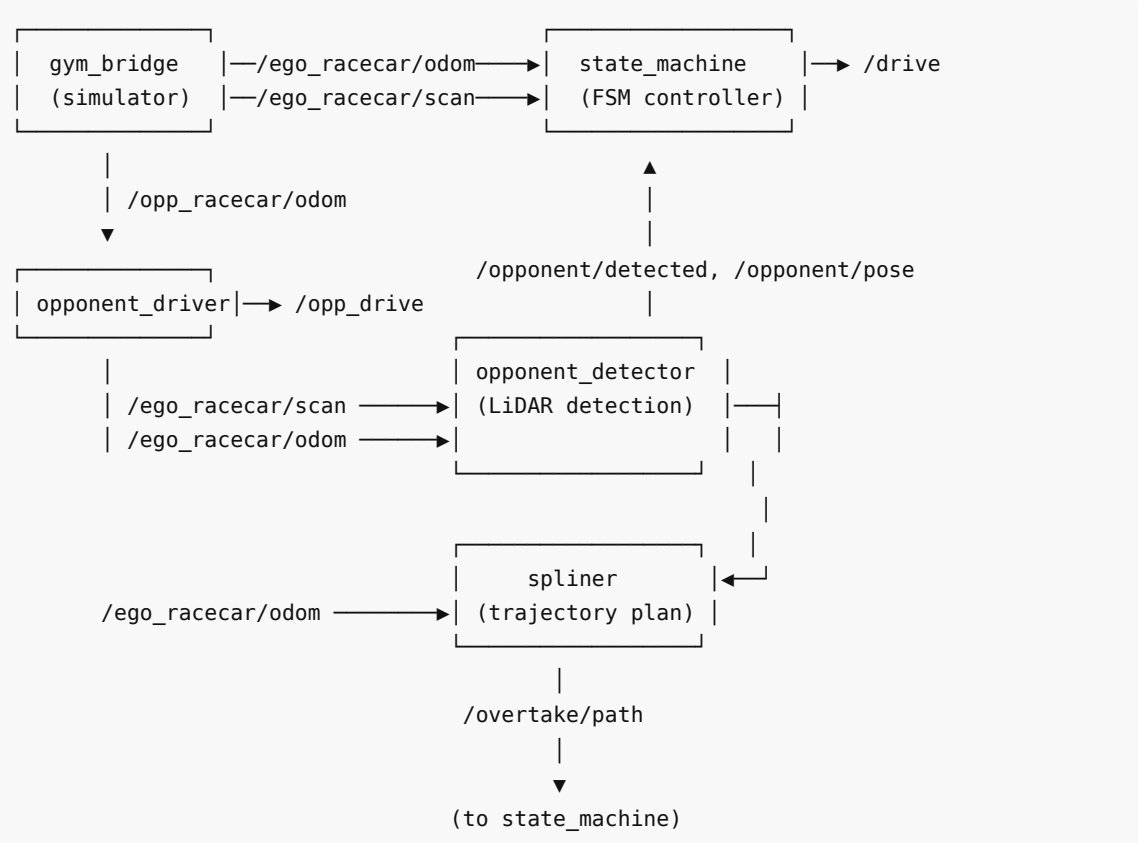
---

7. Head-to-Head Overtaking (⚠️ Work In Progress)

> ⚠️ ***WARNING:*** *The overtaking pipeline runs and the nodes communicate, but end-to-end overtake maneuvers have* ***not been validated****. The car may not successfully pass the opponent. Detection can produce false positives near walls, the spliner path may clip walls on tight sections, and the FSM transition thresholds need real-world tuning.* ***This is the primary area that needs improvement.***

Overview

The overtaking system uses 4 nodes working together:



How to Run

Prerequisite: Set `num_agent: 2` in `sim.yaml` and configure opponent start pose:

```
num_agent: 2
sx1: -9.66
sy1: -2.77
stheta1: 1.84
```

Rebuild: `colcon build --symlink-install`

Then launch in 5 terminals:

Terminal 1 — Simulator:

```
ros2 launch f1tenth_gym_ros gym_bridge_launch.py launch_rviz:=true
```

Terminal 2 — Opponent Driver (drives the opponent car at 50% raceline speed):

```
ros2 run gap_follow opponent_driver_node --ros-args -p speed_scale:=0.5
```

Terminal 3 — Opponent Detector (LiDAR-based detection):

```
ros2 run gap_follow opponent_detector_node
```

Terminal 4 — Spliner (overtake trajectory planner):

```
ros2 run gap_follow spliner_node
```

Terminal 5 — State Machine (replaces stanley_node — controls the ego car):

```
ros2 run gap_follow state_machine_node
```

Terminal 6 — Lap Timer (optional):

```
ros2 run gap_follow lap_timer_node
```

Node Details

Opponent Driver (`opponent_driver_node`)

Simplified Pure Pursuit that drives the opponent car along the same raceline at reduced speed.

- **Subscribes:** `/opp_racecar/odom`
- **Publishes:** `/opp_drive`
- **Key param:** `speed_scale` (default 0.5 = 50% of raceline speed)

Opponent Detector (`opponent_detector_node`) — ⚠️ WIP

LiDAR breakpoint-cluster detection. Works on straights but can produce false positives near walls.

Algorithm:

1. Converts polar LiDAR scan → Cartesian in car-local frame
2. Splits into clusters at range discontinuities (>0.5m jump)
3. Filters by cluster size (3–40 points), distance (<5m, >0.3m), and bounding box (width 0.1–0.5m, depth <0.5m)
4. Validates against raceline proximity (<2m lateral tolerance)
5. Temporal hysteresis: 3 consecutive detections to confirm, 5 misses to clear

Publishes: `/opponent/detected` (Bool), `/opponent/pose` (PoseStamped), `/opponent/markers` (MarkerArray)

Spliner (`spliner_node`) — ⚠️ WIP

Generates cubic-spline overtake trajectories around the detected opponent.

Algorithm:

1. Projects opponent position onto raceline to find arc-length position
2. Uses precomputed distance transform of the map image to determine which side has more wall clearance
3. Creates 5 control points: ego position → approach offset → apex (0.6m lateral offset) → exit offset → raceline return
4. Fits natural cubic spline, samples at 0.1m intervals
5. Speed profile: sinusoidal blend — full raceline speed at edges, 70% at apex
6. Curvature safety check (warns if max curvature > 0.8 rad/m)
7. Publishes path with speed encoded in `pose.position.z`

Publishes: `/overtake/path` (Path), `/overtake/markers` (MarkerArray)

State Machine (`state_machine_node`) — ⚠️ WIP

5-state FSM controller that absorbs Pure Pursuit logic and switches waypoint source based on state:

```
RACELINE → TRAILING → OVERTAKE → RETURN → RACELINE
               ↓                    ↓
          EMERGENCY ← any state with forward LiDAR < 0.8m
```

State	Behavior
`RACELINE`	Normal raceline PP. Enters TRAILING if opponent detected <4m ahead.
`TRAILING`	Follows raceline, speed-caps to match opponent. Enters OVERTAKE when spliner path ready and <3m.
`OVERTAKE`	Follows spliner-generated overtake trajectory. Enters RETURN when opponent is behind or >2m away.
`RETURN`	Follows raceline. Returns to RACELINE when within 0.3m of raceline.
`EMERGENCY`	Hard brake + gap-follow steer toward largest LiDAR opening. Exits when forward range >1.2m.

Publishes: `/drive` , `/state_machine/state` (String), `/state_machine/markers` (MarkerArray)

Known Issues

- **Not able to overtake** gets stuck trailing behind opponent
- **Not reactive** fixed opponent following the a fixed racing line and speed
- **No opponent speed estimation** — the trailing state estimates opponent speed heuristically

---

8. Configuration Reference

Config Files

File	Purpose
`config/sim.yaml`	Simulator: map, LiDAR, poses, agents, timing

`config/slam_toolbox_mapping.yaml`	SLAM Toolbox async mapping config
`config/pure_pursuit.yaml`	Pure Pursuit tuning params
`config/follow_gap.yaml`	Follow-the-Gap reactive planner tuning
`config/race_stack.yaml`	Race state machine config
`config/gamepad_teleop.xbox.yaml`	Xbox controller mapping
`config/rviz/gym_bridge.rviz`	RViz display layout
`config/foxglove/gym_bridge_foxglove.json`	Foxglove Studio layout

Current sim.yaml Defaults

Parameter	Value	Notes
`num_agent`	`2`	Ego + opponent (set to 1 for solo)
`vehicle_params`	`f1tenth`	Uses `ego_racecar.xacro`
`kb_teleop`	`True`	Keyboard teleop enabled in bridge
`use_sim_time`	`False`	Real-time clock
`async_mode`	`True`	Sim steps on timer
`scan_num_beams`	`819`	SICK TIM571 config, 270° FOV
`scan_angle_min/max`	`±135°`	Full 270° LiDAR coverage
`ego start`	`(0, 0, 0)`	Ego spawns at map origin
`opp start`	`(-9.66, -2.77, 1.84)`	~10m ahead on raceline

All Registered Nodes

Node	Package	Entry Point
`gym_bridge`	`f1tenth_gym_ros`	`f1tenth_gym_ros.gym_bridge:main`
`stanley_node`	`gap_follow`	`gap_follow.stanley_node:main`
`pure_pursuit_node`	`gap_follow`	`gap_follow.pure_pursuit_node:main`
`lap_timer_node`	`gap_follow`	`gap_follow.lap_timer_node:main`
`opponent_driver_node`	`gap_follow`	`gap_follow.opponent_driver_node:main`
`opponent_detector_node`	`gap_follow`	`gap_follow.opponent_detector_node:main`
`spliner_node`	`gap_follow`	`gap_follow.spliner_node:main`
`state_machine_node`	`gap_follow`	`gap_follow.state_machine_node:main`

Utility Scripts (workspace root)

Script	Purpose
`generate_raceline.sh`	One-command raceline pipeline
`extract_centerline.py`	CV-based centerline from map images
`speed_profile.py`	Speed profile from curvature + vehicle limits
`visualize_raceline.py`	Raceline visualization (speed color-coded)
`compare_racelines.py`	Compare generated vs reference racelines
`stanley_tuner.py`	Optuna automated parameter optimization